

Practical Implementation of a Quantum String Matching Algorithm

Simone Faro
simone.faro@unict.it
¹Università di Catania
Dipartimento di Matematica e
Informatica
Catania, Italy

Francesco Pio Marino
francesco.marino@phd.unict.it
¹Università di Catania
Dip. di Matematica e Informatica
Catania, Italy
²Univ Rouen Normandie, INSA Rouen
Normandie, Univ Le Havre
Normandie, Normandie Univ
LITIS UR 4108
Rouen, France

Antonio Scardace
¹Università di Catania
Dipartimento di Matematica e
Informatica
Catania, Italy

ABSTRACT

String matching stands as a cornerstone problem in computer science, finding extensive application in many fields, such as data mining, text retrieval, and bioinformatics, just to cite a few. Its essence lies in identifying all occurrences of a specified pattern, typically denoted by length m , within a text of length n . Despite its seemingly straightforward formulation, the problem has sparked a plethora of solutions, stemming from diverse computational paradigms and approaches. Classical solutions exhibit a worst-case time complexity of $\Theta(n)$, contrasting with the capability of quantum computation to achieve a $\tilde{O}(\sqrt{n})$ complexity using Grover search, thereby realizing a quadratic speed-up compared to classical methods. This article presents a first practical implementation of a quantum circuit tailored to address string matching, particularly focusing on binary strings. The exposition delves into the technical intricacies of the implementation, which leverages the Qiskit open-source toolkit developed by IBM. By elucidating various algorithmic nuances overlooked in prior theoretical formulations, our solution serves as a conduit between the realms of text processing and quantum computing, fostering cross-disciplinary dialogue and innovation. While current real-world hardware lacks the capability to execute the proposed circuit, particularly when dealing with lengthy texts, this study presents experimental results from a simulation of the circuit on classical hardware, serving to validate the efficacy of the proposed implementation.

KEYWORDS

string matching, quantum computing, text processing, quantum search

ACM Reference Format:

Simone Faro, Francesco Pio Marino, and Antonio Scardace. 2024. Practical Implementation of a Quantum String Matching Algorithm. In *The 33rd*

International Symposium on High-Performance Parallel and Distributed Computing (HPDC '24), June 3–7, 2024, Pisa, Italy. ACM, New York, NY, USA, 8 pages. <https://doi.org/10.1145/3660318.3660327>

1 INTRODUCTION

Quantum computing stands at the forefront of computer science, employing the principles of quantum mechanics to create highly advanced computing systems that operate in fundamentally different ways from classical computers. Unlike classical computers, which rely on binary bits (0 or 1), quantum computing harnesses quantum bits, or qubits, which can exist in multiple states simultaneously. Moreover, qubits can be entangled, meaning they can perform operations together, leading to faster and more efficient computations compared to classical bits. These unique characteristics provide quantum computers a significant advantage, particularly in tasks such as cryptography and optimization, where they demonstrate remarkable speed and effectiveness.

Quantum computing has led to significant breakthroughs in algorithm development. Notably, Shor's algorithm [22] for integer factorization and Grover's algorithm [13] for unstructured search offer exponential and quadratic speed-ups over classical algorithms, respectively. These algorithms serve as proof-of-concept for the potential of quantum computing, sparking substantial interest in further research and development in this domain. However, it was the recent demonstration of quantum supremacy that ignited widespread interest in quantum computing, prompting the integration of quantum-based solutions across various fields of computer science.

String matching has recently become a focus of interest among researchers in quantum theory. It's a fundamental problem in computer science [11] with applications spanning data mining, text retrieval, image processing, and bioinformatics. Given its broad utility, string matching has emerged as an area of interest in quantum computation [18, 19, 21], where leveraging quantum computers to accelerate pattern matching algorithms could yield significant improvements across various domains.

Formally, given a pattern x of length m and a text y of length n , the exact string matching (ESM) problem may consist in (1) verifying if x occurs in y or in (2) finding all occurrences of x in y and their corresponding positions.



This work is licensed under a Creative Commons Attribution International 4.0 License. QUASAR '24, June 3, 2024, Pisa, Italy
© 2024 Copyright is held by the owner/author(s).
ACM ISBN 979-8-4007-0646-2/24/06.
<https://doi.org/10.1145/3660318.3660327>

Both problems (1) and (2) can be solved in $\Omega(n)$ time in a classical computer [17], with an optimal average time complexity being $\Theta(n \log_{\sigma}(m)/m)$ [9, 25]. Efforts to improve ESM performance in practical scenarios have led to the development of sub-linear algorithms [11]. Notably, the Boyer-Moore-Horspool algorithm [6, 15] deserves mention for its influence on subsequent works [12].

The first quantum solution to ESM, due to Ramesh and Vinay [21], combines Grover search [13] with a parallel string matching technique. They did not provide their solution with a quantum circuit but rather in the *quantum query complexity* model and is designed to solve problem (1) using $\tilde{O}(\sqrt{n})$ queries. Therefore, problem (2) is solved in $\tilde{O}(\sqrt{\rho n})$ time,¹ where ρ is the number of occurrences of x in y , providing real quantum advantage when $\rho = o(n)$. For the average case, Montanaro [18] proved a super-polynomial separation between quantum and classical complexity.

The first solution in the quantum circuit model of computation [25] with a $\tilde{O}(\sqrt{n})$ -time complexity for (1) was presented by Niroula and Nam in 2021 [19]. The details of the corresponding circuit were unfolded and discussed in [8]. The importance of this result is due to the fact that, perhaps, the *circuit-based quantum model* is one of the most important and most widespread quantum models, considering that there are several programming languages characterized by circuit formalism, such as IBM's Qiskit, Microsoft's Q# and Google's Cirq, just to name a few.

In this paper, we introduce the first practical implementation of the Niroula and Nam quantum circuit, focusing on the scenario where the strings involved are binary sequences, to make things simpler. Assuming that the reader is familiar with the basic notions of quantum computing, we delve into the technical intricacies of this implementation, leveraging the open-source toolkit Qiskit, developed by IBM and based on the Python language. Through this study, we address algorithmic aspects overlooked in previous theoretical formulations, thereby offering a valuable contribution to those interested in the practical application of quantum solutions for text processing, beyond the theoretical considerations.

The paper is structured as follows. In Section 2 we give some basic notions while in Section 3 we provide an overview of the Niroula and Nam algorithm. In Section 4, we detail our implementation using Qiskit, discussing various algorithmic considerations and presenting experimental results from circuit simulations on classical hardware, which serve to validate the efficacy of our proposed implementation. Finally, in Section 5, we draw our conclusions.

2 BASICS OF QUANTUM COMPUTATION

We assume in this paper that the reader has some familiarity with quantum computing. However, to make the article self-contained, we introduce in this section the essential notions of quantum computation, useful for understanding our presentation.

Our chosen model of quantum computation in this paper is that of reversible circuits. Quantum circuits are composed of wires carrying qubit values to gates executing elementary operations and facilitate the linear movement of qubits. Input values are inscribed onto the wires from the left side, while output values are read from the right side. Each wire can enter at most one gate at each time

step. Ancillae qubits are often incorporated into the circuit definition to achieve specific computational tasks that would otherwise be unattainable. Complexity measurement in the circuit model may encompass the *number of gates* employed. However, considering the circuit divided into discrete time-steps, where each gate application necessitates one time-step, a more significant measure is the circuit's *depth*, indicating the total number of required time-steps. It is noteworthy that the depth may differ from the total number of gates since gates acting on disjoint qubits can frequently operate in parallel. A simple yet effective example of the aforementioned is the initialization of a quantum register of n qubits to a quantum state characterized by the superposition of all possible 2^n values. This initialization, achievable through n Hadamard operators, each acting on a different qubit, requires constant time as the Hadamard gates are executed in parallel.

Operators in quantum computing are mathematical entities used to represent functional processes that result in the change of the state of a quantum register. Although there are no problems in realizing any quantum operator capable of working in constant time on a quantum register of fixed size, operators of variable size can only be implemented through the composition of elementary gates. We restrict our focus here to enumerating the gates significant to our study. Firstly, the Pauli-X (or X or NOT) gate is the quantum equivalent of the negation gate for classical computers with respect to the standard basis $|0\rangle$ and $|1\rangle$. It operates on a single qubit, mapping $|0\rangle$ to $|1\rangle$ and $|1\rangle$ to $|0\rangle$. The Hadamard (or H) gate is a well-known single-qubit operation that maps the state $|0\rangle$ to $\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$ and $|1\rangle$ to $\frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$, thus creating a superposition of the two basis states with equal amplitudes. The controlled NOT gate (or CNOT) is a quantum logic gate operating on a register of two qubits $|q_0, q_1\rangle$. If the control qubit $|q_0\rangle$ is set to 1, it inverts the target qubit $|q_1\rangle$; otherwise, all qubits remain the same. Formally, it maps $|q_0, q_1\rangle$ to $|q_0, q_0 \oplus q_1\rangle$. The Toffoli gate (also CCNOT gate) is a universal reversible logic gate that works on 3 qubits. If the first two qubits are both set to 1, it inverts the third qubit; otherwise, all bits stay the same. Formally, it maps a 3-qubit register $|q_0, q_1, q_2\rangle$ to $|q_0, q_1, q_0 q_1 \oplus q_2\rangle$. The Swap gate is a two-qubit operator. Expressed in basis states, it swaps the state of the two qubits $|q_0, q_1\rangle$ involved in the operation, mapping them to $|q_1, q_0\rangle$.

Generalizations of the CNOT gate are the n -ary fanout operator and the multiple-CNOT. The former uses a single control qubit and $n - 1$ target qubits; the latter uses $n - 1$ control qubits and a single target qubit. Assuming that the target qubits are all initialized to $|0\rangle$, it is possible, by a divide-and-conquer strategy, to compute fanout in depth $\Theta(\log(n))$ using controlled-not gates and 0 ancillae [10].

The most direct way to implement a multi-controlled NOT is to use the concepts of classical Boolean logic as shown in [23] for classical circuits, and later in [3] in the field of quantum computing. However, this implementation requires $n - 2$ ancillae qubits to store the results of partial computations. In this case the depth of the circuit is linear with respect to the number of control qubits, since parallelism cannot be used. In more recent papers [2, 14], it was shown that a construction rearranging the circuit gates to achieve logarithmic depth by exploiting parallelism is possible, using $n - 2$ and $n/2$ ancillae qubits, respectively.

¹When, in a Grover search, the number of solutions is $\rho > 1$, and we would like to find all of them, $\Theta(\sqrt{\rho n})$ iterations are sufficient and necessary.

3 QUANTUM CIRCUIT BASED ESM

In this Section we briefly outline the quantum algorithm for solving the string matching problem as introduced in [19]. We will provide a concise overview of the structural framework of the quantum algorithm for solving the problem, reserving in-depth discussions on implementation specifics for subsequent sections.

Let y be a binary text of size n , and let x be a binary pattern of size m . The quantum algorithm utilizes a 4-register state $|\psi\rangle = |k\rangle \otimes |a\rangle \otimes |y\rangle \otimes |x\rangle$. The $|y\rangle$ register, with a size of n , contains the characters of the text. Conversely, the $|x\rangle$ register, with a size of m , contains the characters of the pattern. It's important to emphasize that the registers housing information pertaining to the pattern and text are readily accessible to the circuit and do not necessitate initialization. This method of register utilization diverges from prior approaches to the string matching problem [21], which relied on the quantum query model and accessed input solely through oracle queries.

The algorithm also utilizes a register of size $\log(n)$ for containing a text shift value and a register $|a\rangle$ of $\log(n)$ ancillary qubits. The two registers $|k\rangle$ and $|a\rangle$ are both initialized to $|0\rangle^{\otimes n}$.

Figure 1 shows the general circuit implementation of the quantum string matching. The primary operators employed within the circuit consist of the parameterized cyclic rotation operator for a quantum register and the match operator for two quantum registers of equal length. These operators are depicted in Figure 1 as symbols ROT and MATCH, respectively.

The *rotation operator* (denoted by the symbol ROT_j) implements a procedure that cyclically shifts a quantum register of size n by j positions to the left. As a result of this rotation, the qubit at position $(i + j) \bmod (n)$ of the register is moved to position i , where $0 \leq i < n$. Notably, the amplitude of the rotation applied to the register must be specified by an input parameter stored within the quantum register $|j\rangle$ of size $\log(n)$.

The construction of the quantum rotation register was meticulously defined by Pavone and Viola in [20], who also demonstrated that the circuit implementing the operator maintains a depth which is at most $\log(n)$. Subsequent sections will elucidate that the parameterized operator can be realized by composing at most $\log(n)$ rotation operators, resulting in a total complexity of $O(\log^2(n))$.

The *match operator* (denoted by the symbol MATCH) embodies the boolean oracle of the function facilitating comparison between two binary strings of equal length n . Conceptually, the operator accepts as input two quantum registers containing binary strings x and y , along with an output qubit initialized to $|0\rangle$. Its function is to alter the output qubit to $|1\rangle$ only if $x = y$. Symbolically it performs the mapping $\text{MATCH}|x, 0\rangle \rightarrow |x, f(x)\rangle$, where $f(x, y) = 1 \iff x = y$.

In the ensuing sections, we will delineate how this operator can be constructed with logarithmic depth relative to the size of the two strings being compared.

By composing of the rotation and match operators it is possible to construct the *phase oracle of the string matching function* between a pattern x of length m and a text y of length n . More formally, such phase oracle implements the function ϕ , built on x and y , such that $\phi(j) = 1$ if and only if $x = y[j..j + m.1]$. The oracle is obtained in three steps, as shown in Figure 1: (1) rotating the text register by j positions to the left so that the substring $y[j..j + m.1]$

is positioned as a prefix of y ; (2) comparing the string x with the prefix of length m of y ; (3) rotating the text register again so that the original configuration is re-established. The phase oracle U_ϕ performs the mapping $U_\phi|j, x, y\rangle \rightarrow (-1)^{\phi(j)}|j, x, y\rangle$.

Once the phase oracle of the function $\phi(j, x, y)$ has been defined, it is possible to apply Grover's search algorithm to identify one of the possible values of j , if present, for which $\phi(j, x, y) = 1$, answering the question that poses the problem.

It will therefore be necessary to initialize register $|j\rangle$ to the state $|+\rangle^{\log(n)}$ by $\log(n)$ Hadarmard gates and run Grover's algorithm for $\pi/4\sqrt{n}$ iterations before measuring register $|j\rangle$ to identify one of the possible positions where x occurs in y . We recall that each iteration of Grover's algorithm requires the application of the phase oracle of the function ϕ and the Grover Diffuser, which represents a rotation with respect to the state $|+\rangle^{\log(n)}$.

Regarding the time complexity of the resulting algorithm, we recall that the parameterized rotation operator requires $O(\log^2(n))$ time to be executed on a text register of size n , while the match operator requires $O(\log(m))$ time to compare two strings of length m . Thus the phase oracle of the function ϕ can be executed in $O(\log^2(n)) + \log(m) = O(\log^2(n))$, since we assume $m = o(n)$.

The Grover Diffuser requires $O(\log \log(n))$ time, thus each iteration of the Grover's search algorithm requires $O(\log^2(n))$ time. The overall time complexity of the algorithm is therefore $O(\sqrt{n} \log^2(n))$.

Some implementation details, purely inherent to the Grover algorithm, have been left un-outlined in the general description of the algorithm. These include the resolution of the case in which there are no occurrences of the pattern or the case in which there is more than one occurrence and that number is not known a priori.

Lastly, it's noteworthy to mention that our discussion will exclude the representation of ancillary qubits within the circuits, as illustrated in Figure 1. This decision is made for the sake of simplicity, as it enables us to streamline our circuits while also aligning with the practice in Qiskit, where the presence of ancillary qubits is implicitly understood through the application of certain quantum operators.

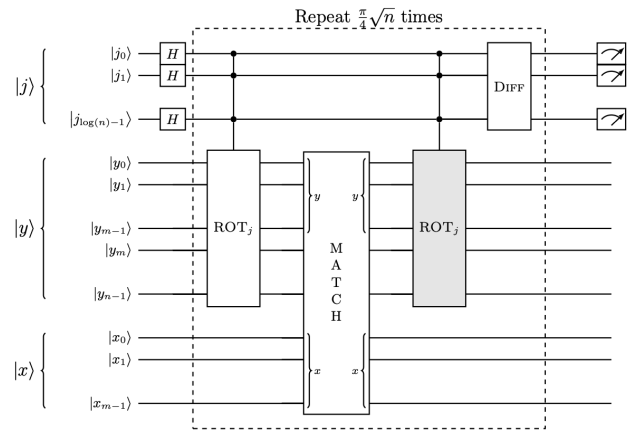


Figure 1: A general circuit implementation of the quantum string matching algorithm [8, 19].

4 A PRACTICAL IMPLEMENTATION

In this section, we provide a comprehensive practical implementation of the ESM algorithm using Qiskit, an open-source toolkit developed by IBM and based on the Python language. Initially, we introduce the fundamental structure of the algorithm and the implementation of the involved operators, as outlined in the reference provided. Subsequently, we delve into more specific algorithmic aspects that enable us to tackle the problem under broader conditions. The code discussed in this study is readily accessible online at: <https://github.com/antonioscardace/Practical-Quantum-ESM>

4.1 The Parameterized Cyclic Rotation Operator

The first operator whose implementation we describe is the circular rotation operator. In this work we will use the implementation carefully described by Pavone and Viola in [20].

A circular shift operator, denoted as R_s , is employed in quantum computing to perform a shift of s positions on a register comprising n qubits. In this work we are interested in applying left rotations to our quantum registers, both for the pattern register and for the text register. This operation ensures that the element at position $(i + s) \bmod n$ is shifted to position i , implementing a circular arrangement for elements beyond the register size n . Formally, the action of the operator R_s is represented by the permutation:

$$|q_0, q_1, \dots, q_{n-1}\rangle \rightarrow |q_s, q_{s+1}, \dots, q_{n-1}, q_0, q_1, \dots, q_{s-1}\rangle$$

For example, rotating a quantum register representing the state $|10110001\rangle$ 3 positions to the left would obtain the new configuration $|10001101\rangle$.

In quantum computing, a rotation operation of a register can be viewed as a permutation of its elements, allowing it to be decomposed into a series of Swap operations. This decomposition typically requires at most n Swaps to transform a sequence into the desired permutation. However, within a quantum circuit, the application of multiple Swap operators can occur concurrently, particularly when dealing with mutually disjoint pairs of qubits.

For a register comprising n qubits, it becomes feasible to execute up to $n/2$ Swap operations simultaneously. This parallel execution facilitates the efficient movement of $n/2$ qubits to their final positions within a single time step. The optimal scenario arises when the register requires rotation precisely by $n/2$ positions, necessitating only $n/2$ parallel Swaps. Specifically, this entails exchanging q_i with $q_{n/2+i}$ for $i = 0..n/2 - 1$. Such an operation exhibits a time complexity of $O(1)$, signifying constant-time execution and exemplifying the efficiency achievable within quantum circuits.

Pavone and Viola present in their work [20] a comprehensive procedure for constructing the circular rotation operator for registers of any size n and any shift amount s , where both n and s are powers of 2. They prove that the resulting circuit exhibits a depth of $O(\log(n))$. Although we refrain from providing a formal proof of this assertion in our current study, it is evident that, in the worst-case scenario, employing $n/2$ swap operations effectively positions a maximum of $n/2$ elements, thus leaving the remaining $n/2$ elements for subsequent positioning. With each subsequent step, the number of qubits requiring accurate positioning decreases by half, enabling a complete rotation in $O(\log(n))$ steps.

```
def rot(n, s):
    qc = QuantumCircuit(n)
    for i in range(1, log2(n)-log2(s)+2):
        for j in range(0, n/(s*(2**i))):
            for q in range(j*s*2**i, s*(j*2**i+1)):
                qc.swap(n-1-(q+s), n-1-(q+2**i*(i-1)*s+s))
    return qc.to_gate(label='ROT'+str(s))
```

```
def rot_gate(n,s):
    return rot(n,s).to_gate(label='Rot'+str(s))
```

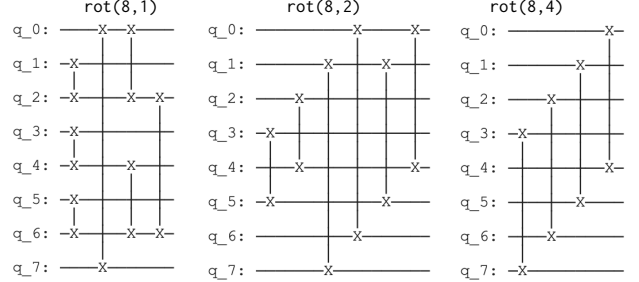


Figure 2: On top: the Qiskit procedure $\text{rot}(n, s)$ for implementing the left-rotation quantum operator which cyclically rotate a register with size n of s positions to the right. On bottom: three leftward rotation operators generated by the procedure for a quantum register of 8 qubits and shift amounts equal to 1, 2 and 4 positions, respectively.

Figure 2 shows the procedure $\text{rot}(n, s)$ for the construction of the left rotation quantum operator together with the circuits generated by the procedure for a quantum register of 8 qubits and shift quantities equal to 1, 2 and 4 positions respectively.

In our implementation we need to apply, in superposition, all possible rotations to the quantum register of size n representing the text. Thus the circular shift operator needs to apply a rotation dependent on an input parameter j . In this context, the operation involves two quantum registers within the circuit: the first register is the parameter $|j\rangle$, which stores the input value related to the rotation amount, with a size of $\lceil \log(n) \rceil$; the second register $|q\rangle$, of size n , represents the register to be rotated. The initialization of $|j\rangle$ is expressed as $|j\rangle = \bigotimes_{i=0}^{\lceil \log(n) \rceil - 1} |j_i\rangle$, where $|j_i\rangle$ is initialized with the i -th least significant bit of the binary representation of k .

Figure 3 showcases the procedural framework, $\text{prot}(n)$, responsible for constructing the parameterized rotation circuit. In the middle, a circuit diagram instantiated by this procedure, exemplified for $n = 8$, is depicted. Notably, the circuit initializes the parameter k using Hadamard operators, thereby representing all feasible shift values between 0 and $n - 1$, in superposition.

For any given value of i , such that $0 \leq i < \log(n)$, the application of the rotation operator $\text{ROT}2^i$ to register $|q\rangle$ is controlled by the qubit $|j_i\rangle$. It is noteworthy that the rotation operator $\text{ROT}2^i$ can be constructed using a circuit with a depth which is, at most, $\log(n)$.

We observe that the possibility that such an operator is controlled by a single qubit, i.e. $|j_i\rangle$, is realized through the application of a technique capable of creating multiple copies of the control qubit and which involves the use of $\log(n)$ ancillary qubits (these ancillae qubits are implicit in the Qiskit implementation).

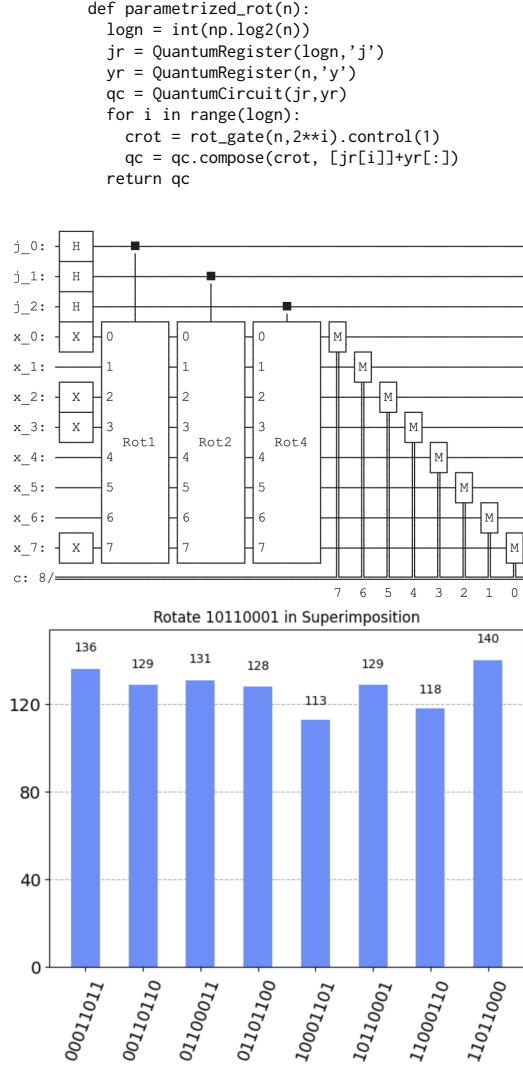


Figure 3: The parameterized cyclic rotation operator. The Qiskit code useful for building the quantum circuit for a string of length n is shown above. In the middle, the circuit generated by the procedure for a string of length $n=8$ is shown. Below, the results of 1000 runs of the circuit using the superimposed rotation parameter are shown.

On the bottom of Figure 3, we show measurements obtained from 1000 runs of the circuit delineated on the left. It's discernible that the resultant alternatives from the measurements correspond to all configurations of the rotated input string 10110001, each bearing an equiprobable likelihood of measurement.

In summary, the parameterized leftward circular shift operator, contingent on the value of an input quantum quantum k , utilizes $n + \lceil \log(n) \rceil - 1$ input qubits, along with $\lceil \log(n) \rceil - 1$ ancilla qubits, exhibiting a depth equal to $O(\log^2(n))$.

4.2 The Match Operator

Once the rotation operator is implemented, the next step involves crafting the binary string matching operator for strings of length n . In quantum computing, given a function $f : \{0, 1\}^n \rightarrow \{0, 1\}$, any quantum operator that maps a register containing the value of a given input $x \in \{0, 1\}^n$ to a register whose value depends on $f(x)$ is called a quantum oracle. Here we are interested in a special kind of quantum oracle, named *Boolean oracle*, which maps the state register $|x, 0\rangle$, to the state register $|x, f(x)\rangle$, for any $x \in \{0, 1\}^n$.

Formally, the match quantum operator serves as a Boolean Oracle for the function $f : \{0, 1\}^n \times \{0, 1\}^n \rightarrow \{0, 1\}$, defined, for any $x, y \in \{0, 1\}^n$ as follows

$$f(x, y) = \begin{cases} 1 & \text{if } x_i = y_i, \forall 0 \leq i < n \\ 0 & \text{otherwise} \end{cases}$$

Given that the comparison between the n different characters of the two strings occurs independently, it lends itself to a circuit of constant depth. However, the validation process to determine whether all characters of the two strings match necessitates the application of a multiple-CNOT operator. This operator can be implemented in logarithmic time relative to the size of the two strings, albeit requiring the utilization of a linear number of ancillary qubits.

The operator resulting from the Qiskit implementation is realized through a circuit of size $2n+1$, assuming that the first n qubits hold the configuration of the first string x , while the subsequent n qubits contain the configuration of the second string y . The final qubit in the circuit serves as the output qubit of the Boolean oracle,

```
def match(n):
    xr = QuantumRegister(n, 'x')
    yr = QuantumRegister(n, 'y')
    out = QuantumRegister(1, 'out')
    qc = QuantumCircuit(xr, yr, out)
    for i in range(n):
        qc.cx(xr[i], yr[i])
        qc.x(yr[i])
        qc.mcx(yr[:], out)
    for i in range(n):
        qc.x(yr[i])
        qc.cx(xr[i], yr[i])
    return qc

def match_gate(n):
    return match(n).to_gate(label='Match'+str(n))
```

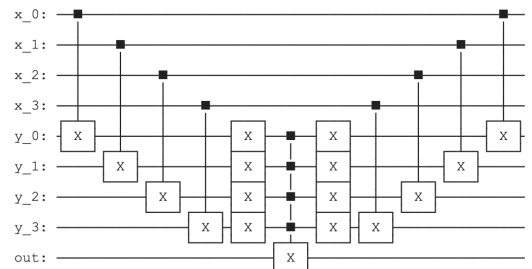


Figure 4: The match operator between binary strings. The Qiskit code useful for building the quantum circuit for strings of length n is shown above. Below, the circuit generated by the procedure for $n=4$ is shown.

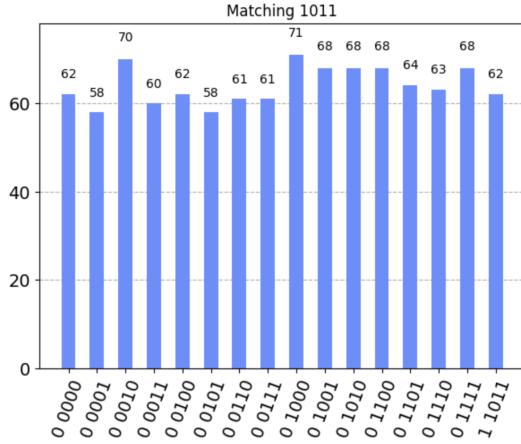


Figure 5: The results of 1000 executions of the match operator using the string $y=1011$ and superimposing the string x .

which will hold the comparison results—yielding 1 if $x=y$ and 0 otherwise.

Figure 4 portrays the Qiskit procedure (at the top) responsible for constructing the aforementioned circuit. The lower section of the same figure illustrates the circuit for $n=4$.

The comparison between characters is based on the following logical relationship, for $0 \leq i < n$

$$x_i = y_i \iff \text{NOT}(\text{XOR}(x_i, y_i))$$

Thus, character comparison is accomplished through a NOT and a CNOT operator (to implement the XOR), while equality between the two strings is verified using a Multiple-CNOT operator. Notably, at the conclusion of the computation, the string registers revert to their initial values via an uncompute process. The entire circuit maintains a logarithmic depth in terms of n .

In Figure 5 the outcomes of 1000 executions of the circuit are showcased, detailing the comparison between a string $y=1011$ of length 4 and a string x of equivalent length. Notably, x encompasses, in superposition, all 16 possible binary strings. From the computation outputs, it becomes evident that only when $x=1011$ is a positive result achieved in the comparison. As anticipated, in all other cases, the comparison yields a resolute 0.

4.3 The Exact String Matching Oracle

We are now poised to amalgamate the operators generated thus far to construct the boolean oracle for the pattern matching operator between pattern and text. The operator fashioned by the `esmo(m, n)` procedure, as depicted in Figure 6, materializes in the form of a Boolean oracle that, post-computation, deposits the outcome within an output register. In the lower part of Figure 6, the Boolean Oracle constructed for a pattern of length 2 and a text of length 4 is displayed. The circuit requires the presence of an additional register: the j register to define the amplitude of the rotation of y . Notably, the uncompute process is executed by reversing the rotation operator, adept at restoring the string y to its initial configuration.

```
def esmo(m,n):
    logn = int(np.log2(n))
    jr = QuantumRegister(logn,'j')
    xr = QuantumRegister(m,'x')
    yr = QuantumRegister(n,'y')
    out = QuantumRegister(1,'out')
    qc = QuantumCircuit(jr,out,xr,yr)
    qc = qc.compose(parametrized_rot(n), jr[:]+yr[:])
    qc = qc.compose(match_gate(m), xr[:]+yr[:m]+out[:])
    qc = qc.compose(parametrized_rot(n).inverse(), jr[:]+yr[:])
    return qc
```

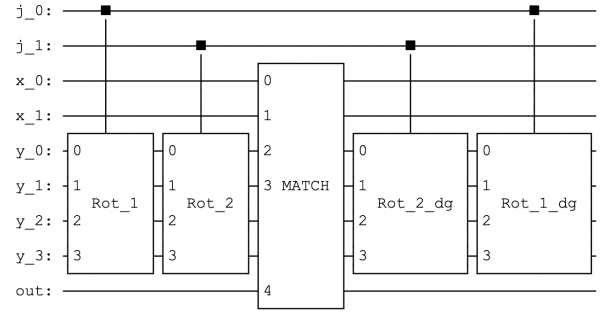


Figure 6: The Boolean oracle for the match between a pattern x of length m and a text y of length n . The Qiskit code useful for building the quantum circuit is shown above. The circuit generated by the procedure for $m=2$ and $n=4$ is shown below.

Assuming that register $|j\rangle$ contains the magnitude of the shift of the text, the *exact string matching boolean oracle* works in four phases: (1) applies a rotation of j positions to the left to the register $|y\rangle$ containing the text, through the application of the parameterized rotation operator; (2) apply the match operator to compare the register $|x\rangle$ with the prefix of length m of the text register $|y\rangle$ and deposit the final result of the output qubit; (3) applies the inverse rotation operator to return the text register $|y\rangle$ to its initial configuration.

Since the parameterized rotation operator takes $O(\log^2(n))$ time and the match operator takes $O(\log(m))$ time, the overall complexity of the Boolean oracle for exact string matching is equal to $O(\log^2(n))$.

In what follows, we indicate the gate that identifies the circuit just discussed through the symbol ESMO.

4.4 Search for a Fixed Number of Occurrences

In this section we define the circuit for solving the string matching problem in the case in which the number of occurrences of the pattern within the text is known a priori. In their solution Niroula and Nam [19] identify one of the occurrences of the pattern using Grover's search algorithm [13].

Grover's search algorithm is one of the most famous results proving theoretical quantum supremacy. It searches a desired item within an unstructured dataset of n items in $O(\sqrt{n})$ time, which is quadratically lower than the lower-bound to the runtime of an algorithm solving the same problem in a classical model of computation. The algorithm is inherently bounded-error, but we know

```

def GroverESM(ESMO,m,n,t):
    logn = int(np.log2(n))
    jr = QuantumRegister(logn,'j')
    xr = QuantumRegister(m,'x')
    yr = QuantumRegister(n,'y')
    out = QuantumRegister(2,'out')
    cr = ClassicalRegister(logn+1,'c')
    qc = QuantumCircuit(jr,xr,yr,out,cr)
    qc.h(jr)
    qc.x(out[0])
    qc.h(out[0])
    iterations = int((np.pi/4)*(np.sqrt(n/t)))
    for i in range(iterations):
        qc = qc.compose(ESMO, jr[:]+xr[:]+yr[:]+out[0])
        qc = qc.compose(Diff, jr[:])
    qc.measure(jr, cr[:logn])
    qc = qc.compose(ESMO, jr[:]+xr[:]+yr[:]+out[1])
    qc.measure(out[1], cr[logn])
    return qc

```

Figure 7: Qiskit procedure for building the Grover's search algorithm for identifying t occurrence of a pattern x of length m within a text y of length n . The implementation assumes that both strings are binary sequences and that the number t of occurrences of x in y is known.

there is no quantum algorithm with less than n queries that solves the problem with certainty for an arbitrary dataset [4].

Given a function $f: \{0,1\}^{\log(n)} \rightarrow \{0,1\}$ with a unique solution $w \in \{0,1\}^{\log(n)}$ such that $f(w) = 1$, the algorithm uses a rotation of the quantum register representing the superposition of all inputs $j \in \{0,1\}^{\log(n)}$ to increase the probability of getting the desired solution w when measuring the register.

Figure 7 shows the Qiskit procedure for building the Grover's algorithm for searching a pattern of length m in a text of length n , assuming that the number t of occurrences is known.

More in detail, the algorithm starts with a register encoding the uniform superposition of all possible inputs $j \in \{0,1\}^{\log(n)}$. Note that the objective register, in which the unique solution w has amplitude 1 and all other items have amplitudes 0, is nearly orthogonal to the starting register. Thus the goal of the algorithm is to apply to the initial register a rotation as close as possible to $\pi/2$ to bring it closer to the objective register. A single iterative step of the algorithm consists of two rotations.

The first one consists in a *phase oracle gate* for the function f , which flips the amplitude of the qubit $|w\rangle$ corresponding to the searched item, leaving the amplitudes of all other inputs unchanged. Such phase oracle is obtained by initializing to the state $|-\rangle$ the output qubit of the ESMO boolean oracle.

The second rotation is performed by the *Grover's Diffusion operator* (identified by symbol Diff), which operates a reflection across $|+\rangle^{\otimes n}$. The overall rotation performed during a single Grover iteration is approximately equal to $2/\sqrt{n}$ radians. Thus, $\pi/4\sqrt{n}$ iterations are necessary and sufficient.

In the case where the function f has exactly t solutions, with $t > 1$, $O(\sqrt{n})$ iterations are still enough to find a solution, but it can be shown that roughly $\frac{\pi}{4}\sqrt{\frac{n}{t}}$ iterations are required [7], and that this bound is optimal [5].

The circuit constructed by the procedure shown in Figure 7, after having applied the Grover's algorithm to search for the text shift and having measured the $|j\rangle$ register, applies a verification procedure by executing the ESMO boolean oracle once and depositing the result in a second output register (out[1]). Formally the output value will be equal to $|1\rangle$ if and only if $x = y[j...j+m-1]$.

Figure 8 shows the Grover's circuit for searching a pattern $x = 00$ in a text $y = 11010011$, with a single occurrence. In this case, Grover's algorithm carries out 2 iterations to converge the initial state $|+\rangle$ to the vector $|w\rangle$. The plot below shows the result of 1000 runs of the circuit, highlighting how the correct solution is identified with a probability greater than 96%.

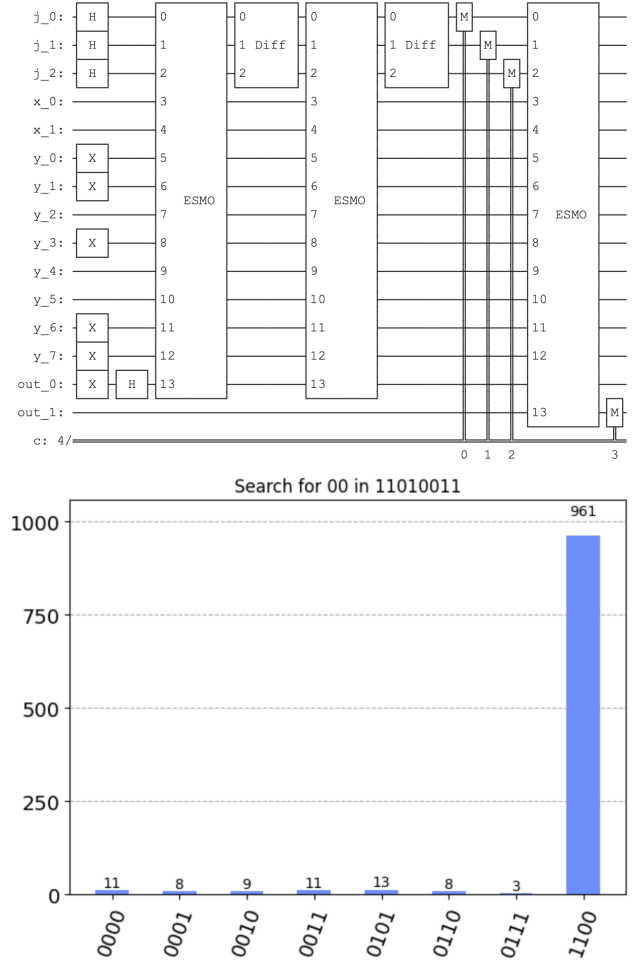


Figure 8: The quantum circuit that implements Grover's search algorithm for solving the string matching problem if the input strings are $x = 00$ and $y = 11010011$, with only one occurrence of x in y . The plot below shows the outputs obtained from 1000 runs of the circuit. The output is a 4-bit binary string, where the first bit represents the result of the verification while the remaining 3 bits represent the value of j identified as the solution to the problem.

```

def search(x, y)
    m = len(x)
    n = len(y)
    for t in range(1, int(n/2)+1):
        results = run(GroverESM(x, y, t), 1)
        results = list(results.keys())[0]
        outcome = int(results[0])
        position = int(results[1:], 2)
        if outcome:
            return position

```

Figure 9: Implementation of the naive iterative search method, which calls adaptively Grover’s algorithm.

4.5 Solving the General Case

In the general case, the value t of occurrences is not known a priori and can only be obtained through a complex procedure based on the Quantum Phase Estimation [16]. Lacking this information precludes the application of Grover’s search algorithm, as the number of iterations hinges solely on its awareness. Since this algorithm works even if a correct t^* estimate of t is known, up to a small multiplicative error $1 + \varepsilon$, such that $1/(1 + \varepsilon) \leq t/t^* \leq 1 + \varepsilon$, several approximate counting solutions have also been proposed [1, 5].

An effective practical solution for this problem has been recently given by Venkateswaran and O’Donnell [24]. It consists in adaptively estimating t by trying all the possible values in the range $[1, 1.1^i]$, for $1 \leq i \leq \log_{1.1}(N/2)$, until a solution is found or, alternatively, responding negatively if no solutions are found. This approach requires a total number of Grover iterations which is still $O(\sum_i \sqrt{N/1.1^i}) = O(\sqrt{N})$. The Qiskit procedure implementing this solution is shown in Figure 9.

For completeness, we also mention a strategy with improved query complexity for large t , which was given by Boyer, Brassard, Høyer, and Tapp [5]. In brief, for $i = 0, 1, 2, \dots, N/2$ it tries a random number of iterations in the range $[1, 1.1^i]$, stopping if ever a solution is found. This algorithm solves the search problem using $O(\sqrt{n/t})$ iterations, despite not knowing t in advance.

5 CONCLUSIONS AND FUTURE WORKS

The primary objective of this study is to offer a comprehensive elucidation of the *Exact String Matching* (ESM) problem within the realm of Quantum Computation. We present an exhaustive exposition of the requisite operators, gates, and the Boolean Oracle involved in the computational circuitry. Subsequently, we delineate two distinct implementations of the ESM problem: the former presupposes a priori knowledge of the number of solutions, whereas the latter dynamically endeavors to estimate the solution count.

In future research, we aspire to explore alternative String Matching algorithms well-established in classical computation, such as parameterized and approximate algorithms. Additionally, another intriguing avenue for exploration would involve investigating methodologies for adaptively estimating the number of solutions.

ACKNOWLEDGMENTS

This work has been partially supported by the National Centre for HPC, Big Data and Quantum Computing, Project CN00000013, affiliated to Spoke 10, co-founded by the European Union.

REFERENCES

- [1] Scott Aaronson and Patrick Rall. 2020. *Quantum Approximate Counting, Simplified*. Society for Industrial and Applied Mathematics, 24–32. <https://doi.org/10.1137/1.9781611976014.5>
- [2] Stefan Balaucă and Andreea Arusoae. 2022. Efficient Constructions for Simulating Multi Controlled Quantum Gates. In *Computational Science - ICCS 2022 (LNCS, Vol. 13353)*. Springer, 179–194. https://doi.org/10.1007/978-3-031-08760-8_16
- [3] Adriano Barenco, Charles H. Bennett, Richard Cleve, David P. DiVincenzo, Norman Margolus, Peter Shor, Tycho Sleator, John A. Smolin, and Harald Weinfurter. 1995. Elementary gates for quantum computation. *Physical Review A* 52, 5 (nov 1995), 3457–3467. <https://doi.org/10.1103/physrevA.52.3457>
- [4] Robert Beals, Harry Buhrman, Richard Cleve, Michele Mosca, and Ronald de Wolf. 2001. Quantum lower bounds by polynomials. *J. ACM* 48, 4 (2001), 778–797. <https://doi.org/10.1145/502090.502097>
- [5] Michel Boyer, Gilles Brassard, Peter Høyer, and Alain Tapp. 1998. Tight Bounds on Quantum Searching. *Fortschritte der Physik* 46, 4–5 (June 1998), 493–505. [https://doi.org/10.1002/\(sici\)1521-3978\(199806\)46:4/5<493::aid-prop493>3.0.co;2-p](https://doi.org/10.1002/(sici)1521-3978(199806)46:4/5<493::aid-prop493>3.0.co;2-p)
- [6] Robert S. Boyer and J. Strother Moore. 1977. A Fast String Searching Algorithm. *Commun. ACM* 20, 10 (1977), 762–772. <https://doi.org/10.1145/359842.359859>
- [7] Gilles Brassard, Peter Høyer, Michele Mosca, and Alain Tapp. 2002. Quantum amplitude amplification and estimation. , 53–74 pages. <https://doi.org/10.1090/conm/305/05215>
- [8] Domenico Cantone, Simone Faro, and Arianna Pavone. 2023. Quantum String Matching Unfolded and Extended. In *Reversible Computation*, Martin Kutrib and Uwe Meyer (Eds.). Springer Nature Switzerland, Cham, 117–133.
- [9] Maxime Crochemore, Artur Czumaj, Leszek Gasieniec, Stefan Jarominek, Thierry Lecroq, Wojciech Plandowski, and Wojciech Rytter. 1994. Speeding Up Two String-Matching Algorithms. *Algorithmica* 12, 4/5 (1994), 247–267. <https://doi.org/10.1007/BF01185427>
- [10] M. Fang, S. Fenner, F. Green, S. Homer, and Y. Zhang. 2006. Quantum Lower Bounds for Fanout. *Quantum Info. Comput.* 6, 1 (jan 2006), 46–57.
- [11] Simone Faro and Thierry Lecroq. 2013. The exact online string matching problem: A review of the most recent results. *ACM Comput. Surv.* 45, 2 (2013), 13:1–13:42. <https://doi.org/10.1145/2431211.2431212>
- [12] Simone Faro, Francesco Pio Marino, and Arianna Pavone. 2020. Efficient Online String Matching Based on Characters Distance Text Sampling. *Algorithmica* 82, 11 (2020), 3390–3412. <https://doi.org/10.1007/s00453-020-00732-4>
- [13] Lov K. Grover. 1996. A fast quantum mechanical algorithm for database search. In *Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing* (Philadelphia, Pennsylvania, USA) (STOC ’96). Association for Computing Machinery, New York, NY, USA, 212–219. <https://doi.org/10.1145/237814.237866>
- [14] Yong He, Mingxing Luo, E. Zhang, Hong-Ke Wang, and Xiao-Feng Wang. 2017. Decompositions of n -qubit Toffoli Gates with Linear Circuit Complexity. *International Journal of Theoretical Physics* 56 (07 2017). <https://doi.org/10.1007/s10773-017-3389-4>
- [15] R. Nigel Horspool. 1980. Practical Fast Searching in Strings. *Softw. Pract. Exp.* 10, 6 (1980), 501–506. <https://doi.org/10.1002/spe.4380100608>
- [16] A. Yu. Kitaev. 1995. Quantum measurements and the Abelian Stabilizer Problem. <https://doi.org/10.48550/ARXIV.QUANT-PH/9511026>
- [17] Donald E. Knuth, James H. Morris Jr., and Vaughan R. Pratt. 1977. Fast Pattern Matching in Strings. *SIAM J. Comput.* 6, 2 (1977), 323–350. <https://doi.org/10.1137/0206024>
- [18] Ashley Montanaro. 2015. Quantum pattern matching fast on average. [arXiv:1408.1816 \[quant-ph\]](https://arxiv.org/abs/1408.1816)
- [19] Pradeep Niroula and Yunseong Nam. 2021. A quantum algorithm for string matching. *npj Quantum Information* 7 (02 2021), 37. <https://doi.org/10.1038/s41534-021-00369-3>
- [20] Arianna Pavone and Caterina Viola. 2023. The Quantum Cyclic Rotation Gate. In *Proceedings of the 24th Italian Conference on Theoretical Computer, 2023 (CEUR Workshop Proceedings, Vol. 3587)*, Giuseppa Castiglione and Marinella Sciortino (Eds.). CEUR-WS.org, 206–218. <https://ceur-ws.org/Vol-3587/4071.pdf>
- [21] H Ramesh and V Vinay. 2003. String matching in $O(n+m)$ quantum time. *Journal of Discrete Algorithms* 1, 1 (2003), 103–110. [https://doi.org/10.1016/S1570-8667\(03\)00010-8](https://doi.org/10.1016/S1570-8667(03)00010-8)
- [22] Peter W. Shor. 1997. Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer. *SIAM J. Comput.* 26, 5 (1997), 1484–1509. <https://doi.org/10.1137/S0097539795293172> [arXiv:https://doi.org/10.1137/S0097539795293172](https://arxiv.org/abs/https://doi.org/10.1137/S0097539795293172)
- [23] Tommaso Toffoli. 1980. Reversible Computing. In *Automata, Languages and Programming (LNCS, Vol. 85)*, J. W. de Bakker and Jan van Leeuwen (Eds.). Springer, 632–644. https://doi.org/10.1007/3-540-10003-2_104
- [24] Ramgopal Venkateswaran and Ryan O’Donnell. 2020. Quantum Approximate Counting with Nonadaptive Grover Iterations. [arXiv:2010.04370 \[quant-ph\]](https://arxiv.org/abs/2010.04370)
- [25] Andrew Chi-Chih Yao. 1979. The Complexity of Pattern Matching for a Random String. *SIAM J. Comput.* 8, 3 (1979), 368–387. <https://doi.org/10.1137/0208029>